

Experiment 07 – Page Object Model (POM) Restructure

1. Module Name and Experiment Title

Module Name: Module 2 – Unit Test Frameworks

Experiment Number: 07

Experiment Title: Page Object Model (POM) Restructure

Programming Language: Python

Automation Framework: Selenium WebDriver

Website: SauceDemo

Website URL: <https://www.saucedemo.com/>

2. Problem Statement

In traditional Selenium automation scripts, web element locators, browser interactions, and test logic are often written together. This can make the code difficult to maintain, reuse, and modify when the application changes.

The objective of this experiment is to restructure a working Selenium login script using the **Page Object Model (POM)** design pattern. Separate page classes are created to manage web elements and user interactions, while test verification remains outside the page classes.

The experiment uses the SauceDemo website to automate the login process and verify that the Products page is displayed successfully.

3. Objective

The main objectives of this experiment are:

1. Understand and implement the Page Object Model design pattern.
2. Create separate page classes for login and inventory page operations.
3. Separate web element locators and UI interaction methods from test logic.
4. Automate the login process using Selenium WebDriver.
5. Verify successful login using an assertion.
6. Capture screenshots at important stages of execution.
7. Improve code readability, reusability, and maintainability.

4. Tools, Software and Concepts Used

Tools and Software:

- Python 3.10
- Selenium WebDriver
- Google Chrome
- Visual Studio Code

Concepts Used:

- Page Object Model (POM)
- Web element locators
- Page classes and methods
- Browser automation
- Explicit waits
- Assertions
- Screenshot capture
- Exception handling using `try-finally`

5. Source Code / Implementation Steps

Source Code

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
import time
import os

os.makedirs("screenshots", exist_ok=True)

class LoginPage:
    def __init__(self, driver):
        self.driver = driver
        self.username = (By.ID, "user-name")
        self.password = (By.ID, "password")
        self.login_button = (By.ID, "login-button")

    def enter_username(self, username):
        self.driver.find_element(*self.username).send_keys(username)

    def enter_password(self, password):
        self.driver.find_element(*self.password).send_keys(password)

    def click_login(self):
        self.driver.find_element(*self.login_button).click()

class InventoryPage:
    def __init__(self, driver):
        self.driver = driver
        self.inventory_title = (By.CLASS_NAME, "title")

    def get_page_title(self):
        return WebDriverWait(self.driver, 15).until(
            EC.visibility_of_element_located(self.inventory_title)
        ).text

driver = webdriver.Chrome()
wait = WebDriverWait(driver, 15)

try:
```

```

driver.maximize_window()
driver.get("https://www.saucedemo.com/")
print("SauceDemo website opened successfully.")
time.sleep(3)

login_page = LoginPage(driver)

driver.save_screenshot("screenshots/01_login_page.png")
print("Screenshot 1: Login page saved.")

login_page.enter_username("standard_user")
time.sleep(3)

login_page.enter_password("secret_sauce")
time.sleep(3)

driver.save_screenshot("screenshots/02_login_credentials.png")
print("Screenshot 2: Login credentials entered.")

login_page.click_login()
print("Login button clicked.")
time.sleep(3)

inventory_page = InventoryPage(driver)
page_title = inventory_page.get_page_title()

assert page_title == "Products", "Login verification failed!"

driver.save_screenshot("screenshots/03_inventory_page.png")
print("Screenshot 3: Inventory page saved.")

print("Page title:", page_title)
print("Page Object Model automation successful!")

finally:
    driver.quit()
    print("Browser closed successfully.")

```

Implementation Steps

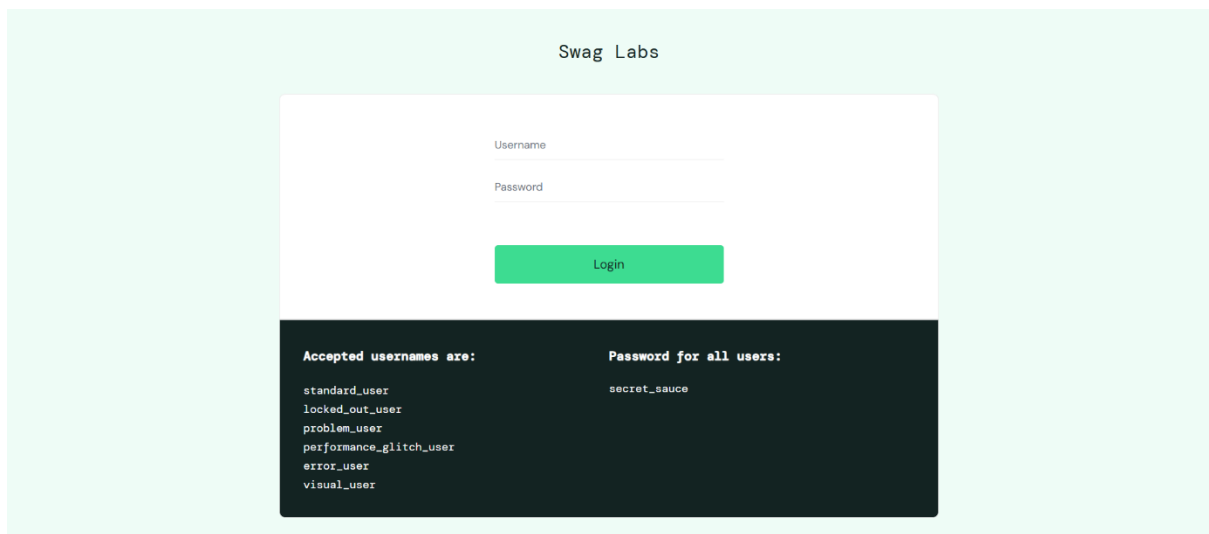
1. Import the required Selenium modules and Python libraries.
2. Create a `LoginPage` class to store the login page locators and methods.
3. Define methods to enter the username, enter the password, and click the login button.
4. Create an `InventoryPage` class to locate and retrieve the Products page title.
5. Initialize the Chrome WebDriver and maximize the browser window.
6. Open the SauceDemo website and capture a screenshot of the login page.
7. Enter the login credentials and capture a screenshot.
8. Click the login button and wait for the inventory page title to appear.
9. Use an assertion to verify that the page title is **Products**.
10. Capture a screenshot of the inventory page and print the execution results.
11. Close the browser automatically using the `finally` block.

6. Output with Clearly Labelled Screenshots

The automation was executed successfully using Python in Visual Studio Code. The following screenshots document the execution.

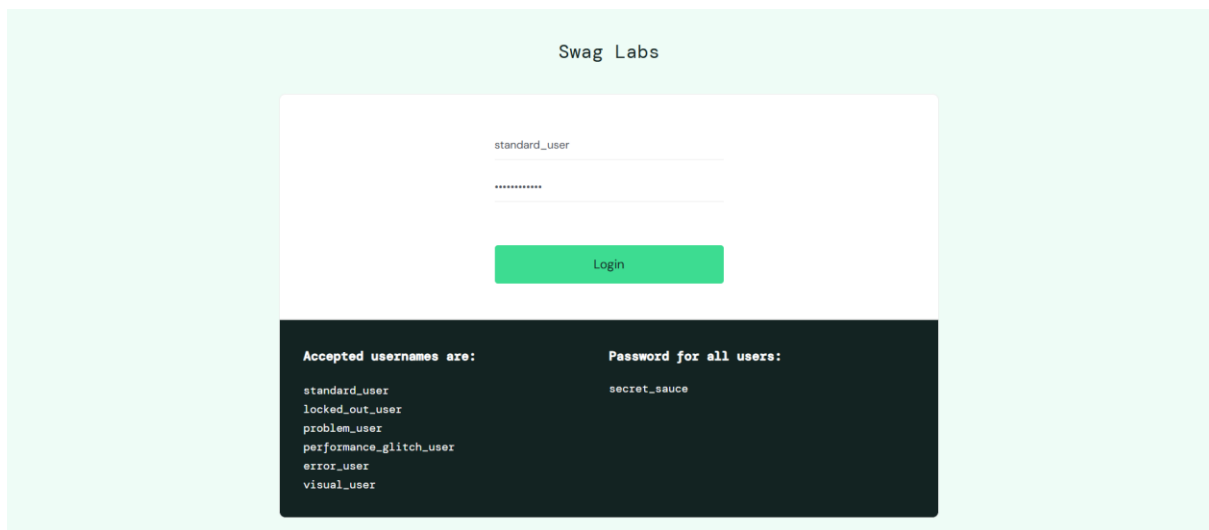
Screenshot 1: Login Page

The SauceDemo login page opened successfully in the Chrome browser.



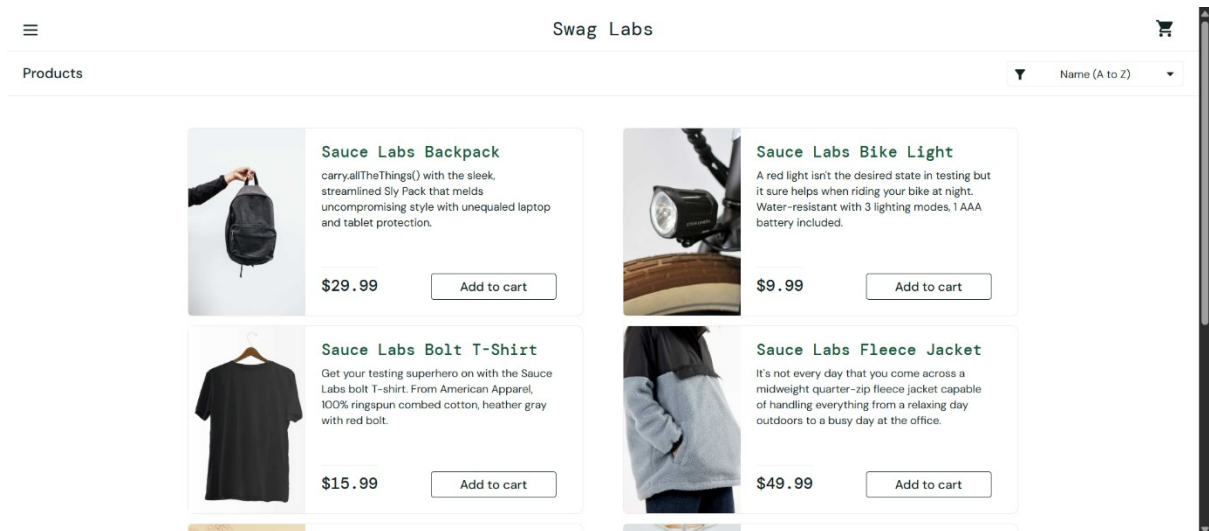
Screenshot 2: Login Credentials Entered

The username and password were entered into the login form before submitting it.



Screenshot 3: Inventory Page

After successful login, the browser navigated to the Products page. The page title was verified as `Products`.

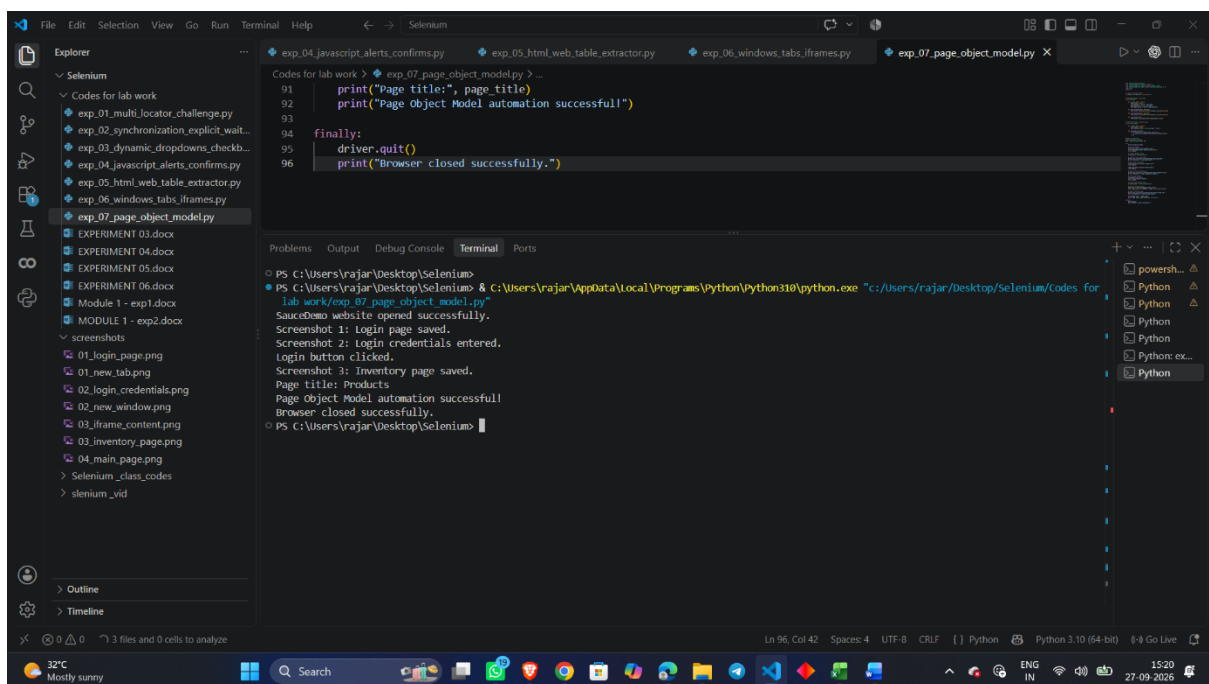


Screenshot 4: Successful Execution and Console Output

The terminal displayed the successful execution messages, including the verified page title and browser closure confirmation.

Console Output:

```
SauceDemo website opened successfully.
Screenshot 1: Login page saved.
Screenshot 2: Login credentials entered.
Login button clicked.
Screenshot 3: Inventory page saved.
Page title: Products
Page Object Model automation successful!
Browser closed successfully.
```



7. Result, Observation and Conclusion

Result:

The Selenium automation script was successfully restructured using the Page Object Model design pattern. The login process was automated, and the application displayed the Products page after successful authentication. The assertion confirmed that the page title was Products.

Observation:

- Separate page classes were used to organize the login and inventory page operations.
- Locators and UI interaction methods were kept inside their respective page classes.
- Test verification was performed outside the page classes.
- Screenshots were captured at important stages of execution.
- The browser closed automatically after the test completed.
- The terminal showed successful execution without any errors.

Conclusion:

The experiment successfully demonstrated the implementation of the Page Object Model in Selenium WebDriver using Python. Separating page-specific operations from test logic makes the automation script more organized, reusable, and easier to maintain.